

A New Model for Software Defect Prediction Using Particle Swarm Optimization and Support Vector Machine

He Can¹, Xing Jianchun¹, Zhu Ruide¹, Li Juelong², Yang Qiliang¹, Xie Liqiang¹

1. PLA University of Science and Technology, Nanjing, China 210007

2. Technical Management Office of Naval Defence Engineering, Beijing, China 100841

Abstract: Software defect prediction could improve the reliability of software and reduce development costs. Traditional prediction models usually have a lower prediction accuracy. In order to solve this problem, a new model for software defect prediction using Particle Swarm Optimization(PSO) and Support Vector Machine(SVM) named P-SVM model is proposed in this paper, which takes advantage of non-linear computing capability of SVM and parameters optimization capability of PSO. Firstly, P-SVM model uses PSO algorithm to calculate the best parameters of SVM, and then it adopts the optimized SVM model to predict software defect. P-SVM model and other three different prediction models are used to predict the software defects in JM1 data set as an experiment, the results show that P-SVM model has a higher prediction accuracy than BP Neural Network model, SVM model, GA-SVM model.

Key Words: software defect prediction, Support Vector Machine, Particle Swarm Optimization, parameters optimization.

1 INTRODUCTION

Software defect^[1] means that software couldn't achieve its function for its own shortcomings. It may cause a catastrophic failure of the whole software system, if it is not detected and modified in time. Accordingly, software defect prediction technique can enable software developers to focus quality assurance activities and allocate effort and resources more efficiently, it in turn can lead to a substantial improvement in software quality. A growing number of studies have been carried out on the subject of software defect prediction using BPNN^[2], Clustering^[3], modified NHPP^[4], and Bayesian model^[5] to realize the prediction. Software defect prediction model based on SVM^[6] has been a hot research direction due to its high prediction accuracy and an excellent capability of non-linear computing. But the research of reference [7] shows that the prediction accuracy of the SVM model is influenced greatly by its parameters, and SVM model usually adopts the trial-and-error method to determine its parameters, which is lacked of guidance in theory.

In order to solve this problem, a new model for software defect prediction using Particle Swarm Optimization(PSO) and Support Vector Machine (SVM) named P-SVM is proposed in this paper. At first, this model uses PSO to optimize the SVM parameters, then using the optimized SVM model to predict the software defect, it provides a guidance in theory to the SVM parameters selection. The results of experiment show that P-SVM model can realize the accurate and fast prediction for software defect.

The rest of paper is organized as follow: section 2 reviews related work, section 3 describes the P-SVM model, section 4 designs the experiment and analyzes the results, section 5 concludes the paper and outlines directions for future work.

2 RELATED WORK

2.1 Support Vector Machine

SVM is a machine learning algorithm based on statistical learning theory and the Structural Risk Minimization Principle (SRM), which is firstly proposed by Vapnik in 1995. It can be used for both classification and regression. The basic idea of SVM is to transform a problem of computing classification function into a quadratic programming (QP) problem with constraints^[8].

If the sample is linear and separable, the process of computing classification function can be converted to the following problem:

$$\begin{aligned} \min \quad & \frac{1}{2} \|w\|^2 \\ \text{subject to} \quad & y_i \left[(w \cdot x_i) + b \right] - 1 \geq 0 \end{aligned} \quad (1)$$

According to the Optimization Theory, the formula above can be transformed into its dual form:

$$\begin{aligned} \min_a \quad & \frac{1}{2} \sum_{i=1}^L \sum_{j=1}^L y_i y_j (x_i \cdot x_j) a_i a_j - \sum_{j=1}^L a_j \\ \text{s.t.} \quad & \sum_{i=1}^L y_i a_i = 0, a_i \geq 0, i = 1, 2, \dots, L \end{aligned} \quad (2)$$

Once the solution has been found in the form of a vector: $a = (a_1 \dots a_n)^T$, the optimal separating hyperplane is found and the classification function is obtained as follow:

$$f(x) = \text{sgn}\{(w * x) + b\} = \text{sgn}\left\{\sum_{i=1}^L y_i * a_i (x_i * x) + b\right\} \quad (3)$$

Usually, the sample set we meet is linear non-separable. In this case, we use the kernel function to address the issue. Kernel function can mapping input data into higher dimensional feature space, then the sample will be separable in this higher dimensional feature space.

This work is supported by National Nature Science Foundation under Grant 70371039

There are many kinds of kernel functions, such as Linear function, Polynomial function, Gaussian function (RBF), Sigmoid function and so on.

This paper chooses the RBF as the kernel function. Parameter γ is very important to RBF, it controls the width of the RBF:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \quad (4)$$

After mapping the data into higher dimensional feature space, there may be still a little amount of sample points which are non-separable. This problem can be solved by introducing positive slack variables ζ and error cost parameter C . ζ is to measure how much the margin constraints are violated and C is to determine the trade-off between the maximization of the margin and minimization of the classification error.

The research of reference [9] shows the performance of SVM model, which uses RBF as its kernel function, is largely dependent on parameter C and γ .

In a word, if the sample set is non-separable, the solution of the classification function is converted to the following problem:

$$\begin{aligned} \min_a \quad & \frac{1}{2} \sum_{i=1}^L \sum_{j=1}^L y_i y_j K(x_i, x_j) a_i a_j - \sum_{j=1}^L a_j \\ \text{s.t.} \quad & \sum_{i=1}^L y_i a_i = 0, C \geq a_i \geq 0, i = 1, 2, \dots, L \end{aligned} \quad (5)$$

And the classification function is achieved as follow:

$$f(x) = \text{sgn} \left\{ \sum_{i=1}^L y_i * a_i * K(x_i * x) + b \right\} \quad (6)$$

2.2 Particle Swarm Optimization

Particle Swarm Optimization^[10] (PSO) comes from simulating the foraging behavior of birds, which is firstly proposed by Kenney in 1995. PSO considers the solution of the problem as a particle in the search space. Let $X_i = (X_{i1}, X_{i2}, \dots, X_{in})$ denote the position of particle i , we can calculate the fitness value of particle i by substituting X_i into fitness function. Resorting to $V_i = (V_{i1}, V_{i2}, \dots, V_{in})$ indicates the velocity of the particle i , Using $P_i = (P_{i1}, P_{i2}, \dots, P_{in})$ denotes the best position which particle i has ever reached, and let $P_g = (P_{g1}, P_{g2}, \dots, P_{gn})$ express the best position which the particle swarm have ever reached. The process of the searching depends on the interact among particle swarm. The formula of velocity and position for the particle i is as following:

$$V_{id}(t+1) = wV_{id}(t) + c_1 \cdot r_1 [P_{id}(t) - X_{id}(t)] + c_2 \cdot r_2 [P_{gd}(t) - X_{id}(t)] \quad (7)$$

$$X_{id}(t+1) = X_{id}(t) + V_{id}(t+1) \quad (8)$$

Where c_1 , and c_2 are two positive constants, r_1 , and r_2 are two random numbers in the range $[0, 1]$, and w is the inertia weight.

PSO, as a new global optimization algorithm, has been applied in parameters optimization and quadratic

programming. So it is available for PSO to optimize SVM parameters.

3 SOFTWARE DEFECT PREDICTION MODEL BASED ON PSO AND SVM

3.1 SVM Parameters Optimization Algorithm Based on PSO

The performance of SVM model, which uses RBF as its kernel function, is largely dependent on parameter C and γ . But SVM model usually adopts the trial-and-error method to determine its parameters, which is lacked of guidance in theory. So we use PSO to optimize the parameters of SVM, in order to improve the accuracy of the prediction model. The main idea of the optimization algorithm is that a pair of SVM parameters are considered as a particle in PSO, particle updates itself using P_i and P_g , unless the algorithm reaches its terminal condition. The index measuring particle i is the accuracy of the prediction model under this pair of parameters. Mean Absolute Percentage Error Function (MAPE) is used as the fitness function.

$$f_{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \dot{y}_i}{y_i} \right| \quad (9)$$

Where y_i and $\dot{y}_i$ are respectively actual value and prediction value of sample i .

The concrete steps of the SVM parameters optimization algorithm based on PSO are as following:

Step1: Setting parameters of PSO, such as particle number, cycle number, c_1 , c_2 , w and so on. Randomly generate initial particles.

Step2: Compute the fitness value of each particle by Eq(9).

Step3: Update P_i and P_g with the fitness value of each particle.

Step4: Each particle moves to its next position using Eq(7), Eq(8).

Step5: Terminal condition checking. If the maximum iteration is met, the cycle stops, output the optimal parameters. Otherwise, turn to step 2.

These steps are illustrated by Figure 1.

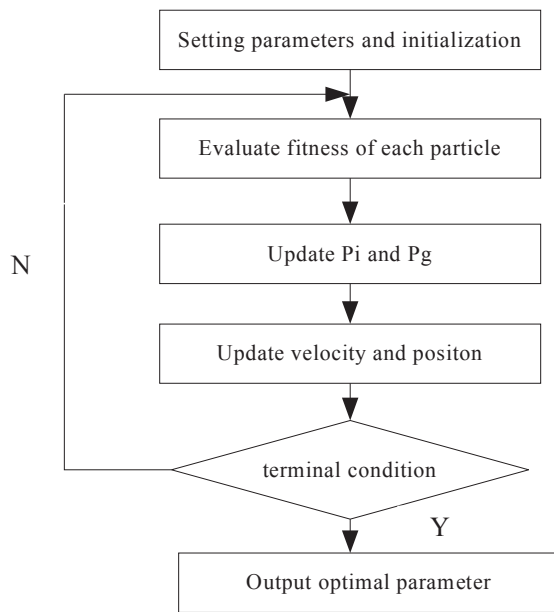


Figure 1 SVM parameters optimization algorithm based on PSO

3.2 Establish of software defect prediction model based on PSO and SVM

On the basis of SVM parameters optimization algorithm above, a new model for software defect prediction named P-SVM is presented in this paper. The steps of the model are as following:

Step1: Achieve the data set.

Step2: Establish the software defect prediction model based on SVM.

Step3: Use SVM parameters optimization algorithm based on PSO to obtain the optimal SVM parameters.

Step4: Resort to optimized SVM prediction model to predict the data set. If the result of prediction reaches the terminal condition, the cycle stops, output the result of software defect prediction. Otherwise, go to step3. These steps are illustrated by Figure 2.

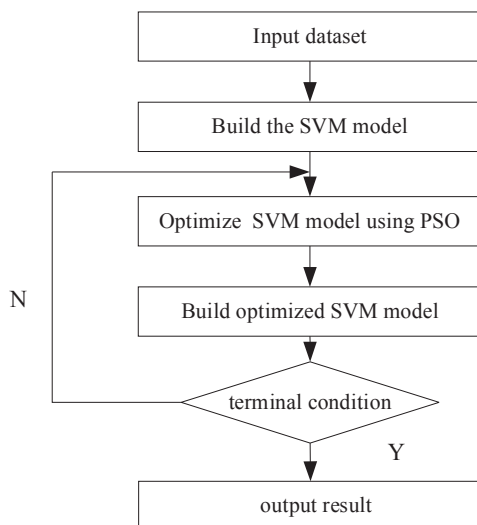


Figure 2 The architecture of P-SVM model

4 EXPERIMENT AND RESULT ANALYSIS

4.1 Experiment Data Set

The JM1 data set^[11] is chosen in this paper for experiment, which comes from a real-time predictive ground system project of NASA. We choose 1000 modules randomly from JM1 as experiment data set, 900 modules for training, 100 modules for testing.

4.2 10-fold Cross Validation

In the experiment, we use 10-fold cross validation procedure where the original sample is arbitrarily partitioned into 10 sub-samples. Of the 10 sub-samples, 9 sub-samples are used as training data whereas a single sub-sample is preserved as the validation data for testing the model. Then, the cross validation process is repeated 10 times, with each of the 10 sub-samples used exactly once as the validation data. The results can be averaged or combined to produce a single estimation.

4.3 Experiment Design

Accuracy is used to measure the performance of prediction models in our work. Accuracy is defined as the ratio of the number of modules correctly predicted to the total number of modules. It can be calculated by the following formula:

$$accuracy = \frac{P}{T} \quad (10)$$

Where P is the number of test modules correctly predicted, T is the total number of test modules.

As mentioned above, an SVM with an RBF kernel function requires the selection of optimal values for parameters C and γ for maximal performance. We use the SVM parameters optimization algorithm based on PSO to realize it, then the pair of parameters values which yield the highest average accuracy are taken as the optimal parameters and used in P-SVM model for classification.

4.4 Experiment Result

Firstly, PSO is used to optimize the SVM parameters. Setting parameters of PSO as follow: particle number $Z=20$, cycle number $N=200$, $c_1=1.5$, $c_2=1.7$.

Choosing GA algorithm as a comparative test. The optimization results of two algorithms is as follow:

Table1 the optimization results of two algorithms

Algorithm	Optimal Parameters
GA	$C=6.544 \gamma=9.134$
PSO	$C=1.193 \gamma=73.541$

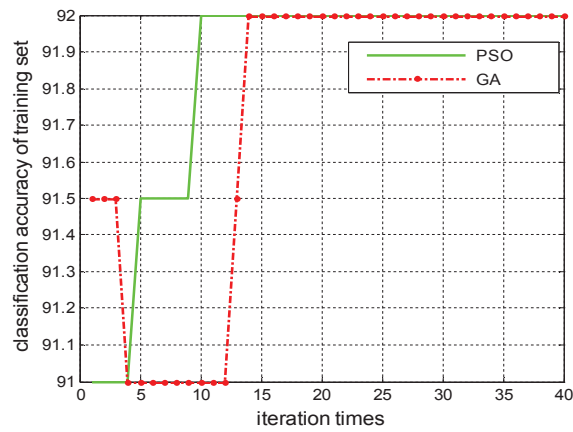


Figure 3 The process of iterative evolution for two algorithms in parameters optimization

After achieving the optimal parameters, P-SVM model is used to predict the defect in testing data set. The result are as follow:

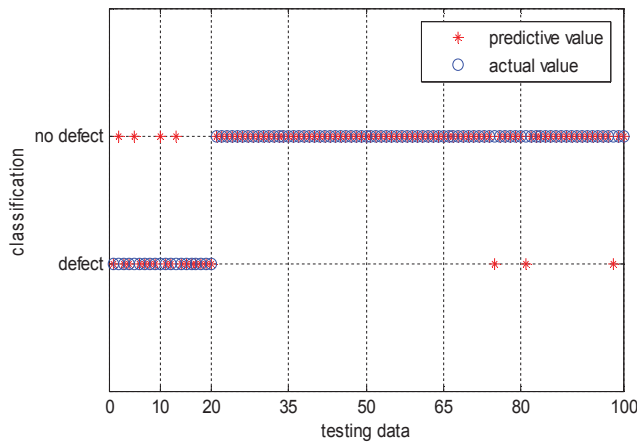


Figure 4 The predictive result of P-SVM model

Figure 4 shows that there are 7 prediction errors of 100 testing data. So the prediction accuracy of P-SVM model is 93%.

4.5 Comparative Experiments

BPNN, SVM and GA-SVM are chosen as comparative experiments. The results are as follow:

Table 2 Accuracy of training data set

Algorithm	BPNN	SVM	GA-SVM	P-SVM
T	900	900	900	900
P	831	813	825	828
Accuracy	92.3%	90.3%	91.7%	92%

Table 3 Prediction accuracy of testing data set

Algorithm	BPNN	SVM	GA-SVM	P-SVM
T	100	100	100	100
P	88	85	89	93
Accuracy	88%	85%	89%	93%

In the table 2-3, P is the number of testing data correctly predicted, T is the total number of testing data.

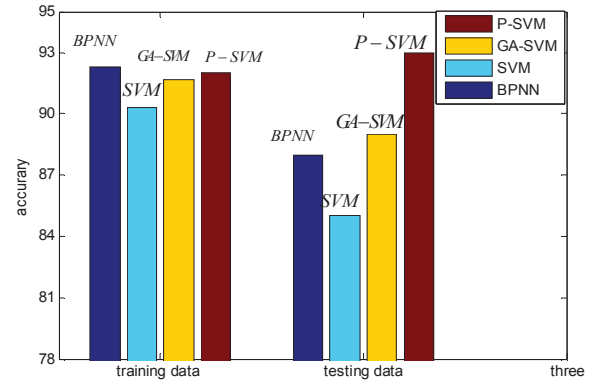


Figure 5 The results of comparative experiments

4.6 Results analysis

From the table 2 and 3, it is seen that the average prediction accuracy of the P-SVM model is 93%, which is 5% higher than BPNN, and 8% higher than SVM model. Comparing with other modified SVM models, P-SVM model is 4% higher than GA-SVM model.

5 CONCLUSIONS AND FUTURE WORK

In this study, a new model for software defect prediction using PSO and SVM named P-SVM model is presented. In this model, the final optimization solution can not be affected by individual particle error, so the model has strong robustness. At the end of the paper, the software defect in public JM1 data set is predicted as an experiment, results demonstrates that P-SVM model has a better prediction accuracy than other kinds of prediction models. Experimental results were only achieved from JM1 data set in this paper, other public data sets and the problems of real world can be tested in the future to verify and extend P-SVM model. And we can also analyze the time complexity of P-SVM model in the future work.

REFERENCES

- [1] Chang R H. Applying Non-Negative Matrix Factorization Based on Fuzzy C-Means to Software Defect Prediction[J]. Journal of Astronautics, 2011, 32(9):2059-2064.
- [2] Donald E Neumann. An Enhanced Neural Network Technique for Software Risk Analysis[J]. IEEE Transactions on Software Engineering, 2002.
- [3] Shi Zhong. Analyzing Software Measurement Data with Clustering Techniques[J]. IEEE Intelligent Systems, 2004.
- [4] Bai. On the Trend of Remaining Software Defect Estimation[J] IEEE Transaction on Systems, Man, and Cybernetics-part A: Systems and Humans, 2008, 38(5): 1129- 1142.
- [5] Fenton. Predicting Software Defects in Varying Development Lifecycles Using Bayesian Nets [J]. Information and Software Technology , 2007, 49(1): 32-43.

- [6] Xing F. A Novel Method For Early Software Quality Prediction Based on Support Vector Machine[C]. Proc. of the 16th ISSRE, 2005, 213-222.
- [7] Wang Q. Software Defect Prediction[J]. Journal of Software (in Chinese), 2008, 19(7):1565-1580.
- [8] Hamdi. Efficient Prediction of Software Fault Proneness Modules Using Support Vector Machines and Probabilistic Neural Network [C]. 5th Malaysian Conference in Software Engineering, 2011, 251-256.
- [9] Karim. Predicting Defect-Prone Software Modules Using Support Vector Machines[J]. Journal of Systems and Software, 2008, 81:649-600.
- [10] Dian. Particle Swarm Optimization: Technique, System and Challenges[J]. International Journal of Computer Applications, 2011, 14(1):19-27.
- [11] NASA metrics data program[DB/OL]. 2005. <http://mdp.ivv.nasa.gov>